

Reflexion Teilaufgabe 3

Ksenia Frank 01577142

June 2023

1 Reflexion

Bei der Betrachtung meiner implementierten Serverlösung stellt sich heraus, dass sie stark von meinem ursprünglichen Plan abweicht. Es wird deutlich, dass ich anfangs keine genaue Vorstellung davon hatte, wie ich das Ganze umsetzen sollte.

Zunächst war mir nicht bewusst, wie man eine Serveranwendung strukturieren sollte und wie man mit Spring Boot und REST arbeitet. In meinem Klassendiagramm hatte ich die "Main"-Klasse als "Server Network" bezeichnet, obwohl sie eigentlich die Klasse mit den Server-Endpunkten darstellen sollte. Darüber hinaus habe ich auch unnötige Methoden wie "startGame()" in die Klasse integriert.

Zweitens hatte ich keine klare Struktur im Hinterkopf, wodurch sich meine Implementierung stark verändert hat. Ich habe meinen Server nach dem Layered-Architecture implementiert und folgende Layers erstellt:

1. Endpoints
2. Controller
3. Services
4. Model

Die Unterteilung eines Spring Boot-Servers in Controller, Services und Model bietet verschiedene Vorteile:

Durch die Verwendung von Controllern verbessert sich die Kohäsion, da die Netzwerkkommunikation zentralisiert wird. Die Controller sind unabhängig von den internen Implementierungsdetails, was zu einer geringen Kopplung führt. Zudem fördern Controller die Datenkapselung und das Information Hiding, indem sie interne Details verborgen halten.

Services zentralisieren die Geschäftslogik und erhöhen dadurch die Kohäsion. Sie ermöglichen eine geringe Kopplung zwischen Controllern und Model-Klassen, da sie unabhängig von spezifischen Implementierungsdetails sind. Services unterstützen ebenfalls die Datenkapselung und das Information Hiding, indem sie interne Details verborgen halten.

Model-Klassen zentralisieren den Datenzugriff und verbessern dadurch die Kohäsion. Sie ermöglichen eine geringe Kopplung zwischen der Geschäftslogik und der Datenbank, da sie unabhängig von spezifischen Implementierungsdetails sind.

Die Aufteilung in Controller, Services und Model verbessert insgesamt die Wartbarkeit, Testbarkeit und Erweiterbarkeit des Systems. Sie ermöglicht eine klare Trennung der Verantwortlichkeiten und fördert eine gute Strukturierung des Codes.

2 Y-Statements

1. Im Kontext der Aufteilung der Serverkomponenten habe ich mich angesichts der Verbesserung der Wartbarkeit, Testbarkeit und Erweiterbarkeit des Systems für die Verwendung von Controllern, Services und Model entschieden und andere Optionen vernachlässigt, um eine klare Trennung der Verantwortlichkeiten zu erreichen. Dies wurde in Kauf genommen, obwohl es anfänglich einen erhöhten Implementierungsaufwand bedeutete. Durch die klare Strukturierung des Codes können zukünftige Änderungen und Erweiterungen leichter umgesetzt werden.
2. Im Kontext der Controller-Komponente habe ich mich angesichts der Verbesserung der Kohäsion, der geringen Kopplung und der Datenkapselung für die Verwendung von Controllern entschieden. Durch die zentrale Verarbeitung von HTTP-Anfragen wird die Struktur und Lesbarkeit des Codes verbessert und die Kommunikation zwischen Client und Server erleichtert.
3. Im Kontext der Service-Komponente habe ich mich angesichts der Verbesserung der Kohäsion, der geringen Kopplung und der Datenkapselung für die Verwendung von Services entschieden und andere Optionen vernachlässigt. Dies wurde in Kauf genommen, obwohl es anfangs einen zusätzlichen Abstraktionslayer und Implementierungsaufwand bedeutete. Durch die Zentralisierung der Geschäftslogik werden die Controller entlastet und die Wiederverwendbarkeit und Testbarkeit der Funktionen verbessert.